Flutter

Dosen	: IKetut Gunawan, S.KOM, M.T.I
Kode/Mata Kuliah	: KB1185/ Aplikasi Mobile
Bobot Mata Kuliah	: 3 Sks
Prodi	: Teknik Informatika
Pertemuan	: 4

Cakupan Materi

- **MaterialApp**

Tujuan pembelajaran materi ini adalah mahasiswa mampu:

1. Mengerti fungsi `MaterialApp`.
2. Memahami hubungan `main()`, `runApp()`, `MaterialApp`, dan `Scaffold`.
3. Mengetahui properti-properti penting yang tersedia di `MaterialApp`.
4. Membuat aplikasi Flutter sederhana dengan judul, tema, halaman awal, dan navigasi.
5. Memahami perbedaan antara konfigurasi aplikasi dan tampilan halaman.

Tujuan Pembelajaran

Rencana Aktivitas Kelas (150 menit)

Waktu	Aktivitas

1 Apa itu MaterialApp ?

`MaterialApp` adalah widget Flutter yang menyediakan konfigurasi dasar untuk aplikasi yang menggunakan Material Design, yaitu sistem desain antarmuka yang dikembangkan Google.

Material Design menyediakan pola tampilan dan interaksi seperti:

- App bar atau header halaman.
- Tombol dan input.
- Warna dan tema aplikasi.
- Navigasi antarhalaman.
- Dialog, menu, dan elemen antarmuka lainnya.

Bayangkan kita sedang membuat aplikasi kampus. Kita perlu menentukan tema aplikasi, halaman yang dibuka pertama kali, dan cara berpindah halaman. `MaterialApp` membantu mengatur hal-hal tersebut.

`MaterialApp dan Scaffold mempunyai tugas berbeda:`

Widget	Tugas utama
`MaterialApp`	`Mengatur konfigurasi dan lingkungan aplikasi`
`Scaffold`	`Menyediakan struktur visual untuk satu halaman`
`AppBar`	`Menampilkan bagian atas halaman`
`Text`	`Menampilkan teks`
`ElevatedButton`	`Menampilkan tombol yang dapat ditekan`


Aplikasi Flutter
Seluruh aplikasi
MaterialApp
Konfigurasi tema, halaman awal, navigasi, bahasa, dan lainnya
Scaffold
Kerangka satu halaman: AppBar, body, FAB, drawer, dan bottom navigation
Widget UI
Text, Icon, Image, Container, Row, Column, Button

2 Memahami code Flutter paling dasar

Berikut contoh aplikasi Flutter sederhana yang dapat dijadikan bahan praktikum.

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

//stl
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Aplikasi Kampus',
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Belajar Flutter'),
        ),
        body: const Center(
          child: Text('Halo Mahasiswa!'),
        ),
      ),
    );
  }
}
```

Menguraikan kode tersebut dari atas ke bawah.

A. `import 'package:flutter/material.dart';`

Baris ini mengimpor library Material Flutter.

Dengan library ini, kita bisa menggunakan widget seperti:

```
MaterialApp()
 Scaffold()
AppBar()
Text()
Icon()
ElevatedButton()
```

```

```

Tanpa import yang sesuai, Dart tidak akan mengenali widget-widget tersebut.

B. `void main()`

```
void main() {
  runApp(const MyApp());
}
```

main() adalah titik awal eksekusi program Dart.

Ketika aplikasi dijalankan, fungsi **main()** dipanggil, kemudian **runApp()** memasang widget utama ke layar Flutter.

C. `runApp()`

```
runApp(const MyApp());
```

`runApp()` menerima widget yang menjadi akar atau root dari antarmuka aplikasi.

Dalam contoh ini, root-nya adalah `MyApp`, dan `MyApp` mengembalikan `MaterialApp`.

D. `class MyApp extends StatelessWidget`

Ini adalah class yang mendefinisikan widget aplikasi kita.

- `class MyApp` adalah nama class.
- `extends StatelessWidget` berarti widget ini tidak menyimpan state yang berubah secara internal.
- `build()` mendeskripsikan widget yang akan ditampilkan.

Pada tahap awal, mahasiswa cukup memahami bahwa `MyApp` merupakan tempat kita menuliskan konfigurasi aplikasi.

E. `return MaterialApp(...)`

Di sinilah konfigurasi utama aplikasi ditulis. Setiap properti di dalam `MaterialApp` mempunyai fungsi masing-masing, yang akan kita bahas berikutnya.

3 Properti yang ada di dalam MaterialApp

Contoh penulisan MaterialApp:

```
MaterialApp(
  title: 'Aplikasi Kampus',
  debugShowCheckedModeBanner: false,
  theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(
      seedColor: Colors.blue,
    ),
  ),
  home: const HomePage(),
)
```

`title`, `theme`, `home`, dan `debugShowCheckedModeBanner` disebut named parameters atau properti konfigurasi yang kita berikan saat membuat widget `MaterialApp`.

Berikut pengelompokan properti yang penting dipelajari. Daftar ini mencakup properti utama dan beberapa properti lanjutan yang tersedia pada `MaterialApp`; tidak semuanya perlu diajarkan di pertemuan pertama.

4 Properti untuk identitas dan halaman awal

4.1 `title`

Menentukan judul aplikasi untuk kebutuhan platform, misalnya pada task switcher atau informasi aplikasi tertentu. Ini bukan judul yang otomatis tampil pada `AppBar`.

```
MaterialApp(
  title: 'Aplikasi Mahasiswa',
  home: const HomePage(),
)
```

Berbeda title judul pada Body/Pages di AppBar ditentukan melalui `AppBar(title: ...)`.

4.2 `home`

Menentukan widget halaman awal aplikasi yang ditampilkan saat aplikasi dibuka, jika tidak ada konfigurasi rute awal yang mengambil alih.

```
MaterialApp(
  home: Scaffold(
    body: Center(
      child: Text('Halaman Beranda'),
    ),
  ),
)
```

Biasanya, halaman awal dibuat dalam class tersendiri seperti HomePage.

4.3 debugShowCheckedModeBanner

Menentukan apakah label DEBUG ditampilkan di sudut kanan atas aplikasi saat debugging.

```
MaterialApp(
  debugShowCheckedModeBanner: false,
  home: const HomePage(),
)
```

Umumnya diatur `false` agar tampilan pengembangan lebih bersih. Ini hanya menghilangkan label, bukan mengubah mode debugging.

4 Properti untuk tema dan tampilan aplikasi

Tema mengatur tampilan yang digunakan oleh widget-widget di dalam aplikasi. Kita dapat mengatur warna, gaya teks, dan tampilan komponen secara konsisten.

5.1 `theme`

`theme` menentukan tema terang atau tema utama yang digunakan aplikasi.

```
MaterialApp(
  theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(
      seedColor: Colors.blue,
    ),
    useMaterial3: true,
  ),
  home: const HomePage(),
)
```

Penjelasan:

- `ThemeData` merupakan konfigurasi tema.
- `ColorScheme.fromSeed()` membuat palet warna dari warna dasar.
- `useMaterial3: true` menggunakan gaya Material 3.

5.2 `darkTheme`

Menentukan tema untuk mode gelap.

```
MaterialApp(
  theme: ThemeData(
    brightness: Brightness.light,
    colorScheme: ColorScheme.fromSeed(
      seedColor: Colors.blue,
    ),
  ),
  darkTheme: ThemeData(
    brightness: Brightness.dark,
    colorScheme: ColorScheme.fromSeed(
      seedColor: Colors.blue,
      brightness: Brightness.dark,
    ),
  ),
  home: const HomePage(),
)
```

5.3 `themeMode`

Menentukan tema mana yang dipakai:

Nilai	Fungsi
`ThemeMode.light`	Selalu memakai tema terang
`ThemeMode.dark`	Selalu memakai tema gelap
`ThemeMode.system`	Mengikuti pengaturan perangkat

```
MaterialApp(
  theme: ThemeData.light(),
  darkTheme: ThemeData.dark(),
  themeMode: ThemeMode.system,
  home: const HomePage(),
)
```

5.4 `color`

Menentukan warna utama yang dapat digunakan sebagai warna representatif aplikasi pada konteks platform tertentu. Properti ini bukan pengganti `theme` atau `colorScheme`.

5.5 Properti tema lanjutan

Beberapa konfigurasi terkait tema lainnya adalah:

- `highContrastTheme` — tema kontras tinggi.
- `highContrastDarkTheme` — tema gelap dengan kontras tinggi.

Keduanya berguna untuk kebutuhan aksesibilitas tertentu dan bukan materi wajib untuk pemula.

Pemahan untuk awal sampai disini untuk properti berikutnya di pertemuan mendatang .

6. Properti untuk navigasi antarhalaman

Ketika aplikasi mulai memiliki halaman Login, Home, Profile, dan Detail, `MaterialApp` menyediakan konfigurasi untuk navigasi.

6.1 `routes`

`routes` adalah kumpulan rute bernama (named routes) yang memetakan nama halaman ke widget.

6.2 `initialRoute`

Menentukan nama rute awal yang akan dibuka oleh sistem navigasi.

6.3 `onGenerateRoute`

Digunakan untuk membuat rute secara dinamis, misalnya ketika halaman membutuhkan parameter.

6.4 `onUnknownRoute`

Menentukan rute cadangan apabila rute yang diminta tidak ditemukan oleh mekanisme navigasi yang relevan.

6.5 Properti navigasi lanjutan

- `navigatorKey` — key untuk mengakses `Navigator` tanpa bergantung langsung pada `BuildContext`.
- `navigatorObservers` — daftar observer untuk mengamati peristiwa navigasi.
- `onNavigationNotification` — callback untuk menerima notifikasi navigasi.

Properti ini biasanya dipelajari setelah mahasiswa memahami `Navigator.push()` dan `Navigator.pop()`.

7. Properti untuk bahasa dan lokalisasi

Jika aplikasi akan mendukung Bahasa Indonesia, Bahasa Inggris, atau bahasa lain, `MaterialApp` menyediakan properti untuk lokalisasi.

7.1 `locale`

Menentukan locale yang digunakan aplikasi, misalnya Bahasa Indonesia.

7.2 `supportedLocales`

Menentukan daftar bahasa yang didukung aplikasi.

7.3 `localizationsDelegates`

Mendaftarkan delegate yang menyediakan resource terjemahan dan konfigurasi lokal untuk widget.

7.4 `localeResolutionCallback`

Callback untuk menentukan locale yang dipakai berdasarkan locale yang diminta.

7.5 `localeListResolutionCallback`

Serupa dengan `localeResolutionCallback`, tetapi mempertimbangkan daftar locale yang disukai oleh perangkat.

7.6 `onGenerateTitle`

Function untuk menghasilkan judul aplikasi secara dinamis, misalnya berdasarkan bahasa yang sedang digunakan.

8. Properti untuk membangun tampilan secara global

Ada beberapa properti yang memengaruhi widget di seluruh aplikasi, bukan hanya satu halaman.

8.1 `builder`

Memungkinkan kita membungkus navigator atau widget utama aplikasi dengan widget lain.

8.2 `scrollBehavior`

Mengatur perilaku scrolling secara global, misalnya untuk menyesuaikan cara pengguna menggulir daftar pada perangkat tertentu.

8.3 `shortcuts`

Mendaftarkan pintasan keyboard untuk aksi tertentu.

8.4 `actions`

Memetakan jenis intent atau aksi ke handler yang akan menjalankannya.

Contoh penerapan `MaterialApp` secara lengkap

Sekarang kita gabungkan properti yang paling relevan untuk mahasiswa pemula: `title`, `theme`, `debugShowCheckedModeBanner`, `home`, `Scaffold`, dan `AppBar`.

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Aplikasi Kampus',

      debugShowCheckedModeBanner: false,

      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.indigo,
        ),
        useMaterial3: true,
      ),

      home: Scaffold(
        appBar: AppBar(
          title: const Text('Belajar Flutter'),
        ),
        body: const Center(
          child: Text(
            'Selamat Belajar Flutter!',
            style: TextStyle(fontSize: 20),
          ),
        ),
      ),
    );
  }
}
```

Latihan praktikum mahasiswa

Latihan 1 — Identitas aplikasi

Ubah title menjadi 'Aplikasi Mahasiswa' dan teks di tengah menjadi nama masing-masing.

Latihan 2 — Tema warna

Ganti seedColor dari Colors.indigo menjadi Colors.green, Colors.orange, atau warna lain.

Latihan 3 — AppBar

Ubah judul AppBar menjadi 'Dashboard Mahasiswa'.

Latihan 4 — Tema gelap

Tambahkan darkTheme dan ubah themeMode menjadi ThemeMode.dark.

Latihan 5 — Halaman profil

Buat class ProfilePage dan gunakan sebagai halaman yang dibuka melalui home.

Kesimpulan

Ingat tiga konsep utama berikut.

1. `MaterialApp` adalah konfigurasi utama aplikasi Flutter, termasuk tema dan navigasi.
2. `Scaffold` adalah kerangka visual untuk satu halaman, seperti tempat `AppBar` dan `body` diletakkan.
3. Widget adalah komponen penyusun tampilan, seperti `Text`, `Icon`, `Container`, dan `ElevatedButton`.